SIMATS ENGINEERING (SAVEETHA SCHOOLOFENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL -2

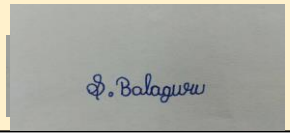
Short Answer Test

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO1**	**Assessment**	**Assessment Tool 2: Short Answer Test**
Weight age	**30% of CIA**	**Total Marks**	**100**
CO1Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	**Balaguru S**	**Reg.No.**	**192312070**
Section/Batch	**Slot A / 2023**	**Date**	**22-10-2026**

Code of Conduct:

I, Balaguru S, certify that this submission is my original work and that I have adheredtotheguidelinesspecifiedforthisassessment.Iunderstandthatanyviolationofacademicintegrity ruleswillresultindisciplinaryaction.

Signature of the student:



1. A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

1. Testing as an Engineering Activity

Introduction:

Testing as an engineering activity is a systematic, planned, and structured process used to identify defects and improve software quality. It replaces random testing with a scientific approach that ensures reliable and consistent results.

Redesigning the Testing Approach:

1. **Requirement Analysis:** Identify functional and non-functional requirements and understand user expectations.
2. **Test Planning:** Define testing objectives, scope, resources, schedule, and responsibilities.
3. **Test Case Design:** Use equivalence partitioning, boundary value analysis, and decision table testing to cover normal, invalid, and boundary inputs.
4. **Systematic Execution:** Execute test cases and compare actual results with expected results.
5. **Defect Reporting:** Record defects with reproduction steps, severity, expected results, and actual results.
6. **Regression Testing:** Retest the software after fixes to ensure existing functionality remains unaffected.
7. **Test Evaluation:** Measure effectiveness using test coverage, defect detection rate, and defect density.

How Structured Test Design Improves Quality:

- **Better Coverage:** Ensures all important requirements and scenarios are tested.
- **Early Detection:** Identifies defects early, reducing correction costs.
- **Consistency:** Produces repeatable and reliable testing results.
- **Fewer Missed Defects:** Covers valid, invalid, and boundary conditions systematically.
- **Improved Efficiency:** Reduces duplicate testing and saves time and resources.
- **Higher Reliability:** Regression testing prevents existing features from breaking after modifications.

Example:

For a login page, random testing may check only valid credentials. Structured testing checks valid usernames, incorrect passwords, empty fields, boundary-length inputs, and repeated login failures. If an incorrect password allows access, the tester documents the defect, and the developer fixes it. Regression testing then verifies that login functions correctly.

Conclusion:

Testing as an engineering activity replaces random testing with a systematic and measurable process. Structured test design improves coverage, consistency, defect detection, and software reliability while reducing development costs and increasing customer satisfaction.

2. A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Introduction:

A well-defined testing process is a systematic approach involving test planning, test design, execution, defect reporting, and evaluation. In banking systems, it ensures secure transactions, accurate data processing, high reliability, and continuous service availability.

Steps to Standardize the Testing Process:

1. **Requirement Analysis:** Identify requirements for account management, fund transfers, authentication, and transaction processing.
2. **Test Planning:** Define testing objectives, resources, schedules, and responsibilities.
3. **Test Case Design:** Prepare test cases for valid transactions, invalid inputs, insufficient balances, and concurrent transactions.
4. **Test Execution:** Execute functional, security, performance, and integration tests systematically.
5. **Defect Reporting:** Record failures, identify their causes, and prioritize corrections based on severity.
6. **Regression Testing:** Retest the system after fixes to ensure existing banking functions remain unaffected.
7. **Test Evaluation:** Measure test coverage, defect rates, and system performance before deployment.

Benefits with Relevant Examples:

- **Improved Accuracy:** Testing verifies that account balances and transaction amounts are calculated correctly. For example, a fund transfer of ₹1,000 must debit and credit the correct accounts.
- **Enhanced Security:** Authentication and authorization testing identify vulnerabilities. For example, unauthorized users must not access another customer's account.
- **Higher Reliability:** Failure and recovery testing ensures transactions are not duplicated or lost during system failures.
- **Better Performance:** Load testing verifies that the banking system handles thousands of simultaneous transactions without excessive delays.
- **Data Integrity:** Database testing ensures account records remain consistent during deposits, withdrawals, and transfers.
- **Reduced Maintenance Costs:** Early defect detection prevents expensive failures after deployment.
- **Continuous Quality Improvement:** Regression testing ensures that software updates do not introduce new defects.

Example:

If a network failure occurs during a fund transfer, an untested banking system may debit the sender without crediting the receiver. A standardized testing process includes failure-recovery and transaction-integrity test cases to verify that the transaction is completed correctly or rolled back safely. This prevents financial discrepancies and improves customer trust.

Conclusion:

A well-defined testing process improves banking software quality through systematic planning, accurate verification, security testing, performance evaluation, and regression testing. It reduces failures, protects financial data, ensures transaction integrity, and provides secure, reliable, and uninterrupted banking services.

3. During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Introduction:

Software testing in the Software Development Life Cycle (SDLC) is a systematic process of verifying and validating software at every development stage. A structured testing process helps identify defects early, prevents integration failures, and improves overall software quality.

Testing Process in SDLC:

1. **Requirement Analysis:** Identify functional and non-functional requirements of the e-commerce platform, such as login, product search, cart, payment, and order tracking. Prepare testable requirements.
2. **Test Planning:** Define testing objectives, scope, resources, tools, schedule, and responsibilities. Select appropriate testing techniques for each development phase.
3. **Test Case Design:** Prepare test cases, test data, and expected results. Include valid and invalid inputs, boundary conditions, and error scenarios.
4. **Unit Testing:** Test individual modules independently. For example, verify that the shopping cart correctly calculates product prices, quantities, taxes, and discounts.
5. **Integration Testing:** Test interactions between modules. For example, verify that the cart correctly communicates with the payment gateway and order management system.
6. **System Testing:** Test the complete application against specified requirements. Verify product browsing, login, checkout, payment processing, and order confirmation.
7. **Acceptance Testing:** Confirm that the application meets customer and business requirements. For example, verify that customers can successfully purchase products and receive order confirmations.
8. **Regression Testing and Maintenance:** Retest existing features after defect fixes or updates to ensure that changes do not introduce new defects.

How Process Discipline Ensures Better Quality:

- **Early Defect Detection:** Identifies errors before they spread to later development stages.
- **Better Integration:** Verifies communication between modules and prevents interface failures.
- **Consistent Testing:** Standard procedures ensure repeatable results across all development phases.
- **Improved Test Coverage:** Ensures that all important requirements and scenarios are tested.
- **Reduced Cost:** Fixing defects early reduces debugging and maintenance expenses.
- **Higher Reliability:** Regression testing ensures that existing features continue to work correctly.
- **Better Documentation:** Test cases and defect reports improve communication among developers and testers.

Example:

In an e-commerce platform, the payment module may work correctly during unit testing but fail to update the order status during integration. Structured integration testing identifies this problem before release. System testing then verifies the complete purchase process, while regression testing ensures that subsequent fixes do not affect existing functionality.

Conclusion:

Testing as a structured SDLC process ensures that software is tested systematically from requirements analysis to maintenance. Process discipline improves defect detection, integration, test coverage, and reliability while reducing development costs and delivering a high-quality e-commerce platform.

4. A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Introduction:

In software testing, error, defect, and failure are related terms used to identify software problems. Understanding these terms helps developers identify root causes and improve software quality.

Basic Testing Definitions:

1. **Error:** A human mistake made during software development, such as writing an incorrect condition.
2. **Defect:** A flaw in the software caused by an error in requirements, design, or coding.
3. **Failure:** Occurs when software behaves incorrectly during execution and produces an unexpected result.

Relationship Between the Terms:

An error made by a developer may introduce a defect into the software. When that defect is executed under specific conditions, it may cause a failure.

Error → Defect → Failure

Example: Login System

- **Error:** The developer mistakenly writes a condition that accepts an incorrect password.
- **Defect:** The login code contains an incorrect password-validation condition.
- **Failure:** A user enters an incorrect password but successfully logs in instead of receiving an error message.

Classification of the Login Issue:

If the expected output is “Login failed” but the actual output is “Login successful,” the observed problem is a **failure**. The incorrect validation logic is the **defect**, and the developer's coding mistake is the **error**.

Importance of Understanding These Terms:

- **Accurate Reporting:** Helps testers describe problems clearly.
- **Root Cause Analysis:** Helps developers identify and correct the original mistake.
- **Improved Quality:** Prevents similar problems from recurring.

Conclusion:

An error is a human mistake, a defect is a flaw in the software, and a failure is incorrect behavior observed during execution. Identifying these differences supports effective debugging and improves software reliability.

5. A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

5. Early Testing Principle in Software Testing

Introduction:

The principle of early testing states that testing activities should begin as early as possible in the Software Development Life Cycle (SDLC). Detecting defects in the initial stages reduces correction costs and improves software quality. It emphasizes prevention rather than correcting defects after development. Testing should be a continuous activity throughout the development process. Early testing also helps identify misunderstandings in requirements before they affect the design and coding stages.

Applying the Early Testing Principle:

1. **Requirement Review:** Check requirements for ambiguity, inconsistency, and missing information.
2. **Design Review:** Examine software architecture and design documents to identify potential defects.
3. **Early Test Case Preparation:** Prepare test cases alongside requirement and design development.
4. **Unit Testing:** Test individual components during coding to identify errors immediately.
5. **Continuous Testing:** Perform integration and regression testing throughout development.

How Early Testing Improves Effectiveness:

- **Reduced Cost:** Defects found early are easier and cheaper to fix.
- **Faster Detection:** Identifies problems before they spread to other modules.
- **Better Quality:** Prevents defects from reaching later testing stages.
- **Improved Productivity:** Reduces repeated debugging and rework.
- **Lower Risk:** Minimizes the possibility of critical failures after release.
- **Better Planning:** Helps teams allocate testing resources efficiently.
- **Improved Customer Satisfaction:** Reduces the chance of users encountering defects in the final product.

Example:

Consider an e-commerce application where the discount calculation requirement is incorrectly defined. If identified during requirement review, it can be corrected with minimal effort. If discovered during system testing, developers may need to modify several modules. If discovered after deployment, it may cause incorrect customer bills, financial losses, and expensive software updates. Early testing prevents these problems and reduces maintenance costs.

Conclusion:

The early testing principle improves testing effectiveness by identifying and correcting defects at the earliest possible stage. It reduces development costs, prevents defect propagation, saves time, and improves overall software reliability. Regular reviews and continuous testing also improve communication between developers and testers. Therefore, integrating testing into every SDLC phase helps deliver dependable, high-quality software.

6. During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Introduction:

Requirement analysis is the initial phase of the Software Development Life Cycle (SDLC), where user needs are identified and documented. Testers contribute by reviewing requirements to identify ambiguities, inconsistencies, and missing information before development begins.

Tester's Contributions During the Requirement Phase:

1. **Requirement Review:** Examine requirements to identify unclear, incomplete, or conflicting statements.
2. **Clarification:** Discuss doubts with developers, customers, and stakeholders to establish clear requirements.
3. **Testability Assessment:** Ensure every requirement is measurable, verifiable, and testable.
4. **Test Case Preparation:** Develop initial test scenarios based on functional and non-functional requirements.
5. **Risk Identification:** Identify critical healthcare functions, such as patient records, prescriptions, and medical reports.
6. **Requirement Validation:** Confirm that documented requirements meet user needs and regulatory expectations.

Importance of Early Testing Involvement:

- **Early Defect Detection:** Identifies requirement errors before they affect design and coding.
- **Reduced Cost:** Fixing requirement defects early is less expensive than correcting implementation defects.
- **Improved Quality:** Ensures the system meets its intended requirements.
- **Better Communication:** Reduces misunderstandings among stakeholders, developers, and testers.
- **Patient Safety:** Helps prevent errors in sensitive functions, such as medication dosage and patient identification.

Example:

In a healthcare system, the requirement "display patient records quickly" is ambiguous because it does not specify a response time. A tester can request a measurable requirement, such as displaying records within two seconds under defined conditions. This makes the requirement testable and prevents disagreements during system testing.

Conclusion:

Early tester involvement improves requirement clarity, prevents defects from spreading to later development stages, and reduces correction costs. In healthcare systems, it also supports patient safety, data accuracy, and software reliability.

7. A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Introduction:

Software defects can originate at any stage of the Software Development Life Cycle (SDLC). Unclear or ambiguous requirements often lead to incorrect design decisions, coding mistakes, and failures during software execution.

How Defects Originate in the Software Lifecycle:

1. **Requirement Phase:** Incomplete or ambiguous requirements create misunderstandings about expected system behavior.
2. **Design Phase:** Developers may design incorrect workflows or architectures based on misunderstood requirements.
3. **Coding Phase:** Incorrect interpretations are translated into faulty program logic.
4. **Testing Phase:** Inadequate test cases may fail to identify defects caused by unclear requirements.
5. **Deployment Phase:** Undetected defects reach users and produce incorrect outputs or system failures.

How Requirement Clarity Prevents Defects:

- **Clear Specifications:** Define inputs, outputs, calculations, and expected behavior precisely.
- **Stakeholder Reviews:** Confirm that requirements match business needs.
- **Acceptance Criteria:** Establish measurable conditions for determining whether requirements are satisfied.
- **Requirement Traceability:** Link requirements to design, code, and test cases to ensure complete coverage.
- **Early Validation:** Review requirements before implementation to identify inconsistencies and missing details.

Example:

In a payroll system, the requirement “calculate overtime pay correctly” does not specify the overtime rate or calculation method. A developer may use an incorrect rate, resulting in inaccurate salaries. A clear requirement specifying the overtime rate, eligible hours, and calculation formula allows developers to implement the correct logic and testers to verify the output.

Conclusion:

Defects may originate from unclear requirements and propagate through design, coding, and testing. Clear specifications, stakeholder reviews, measurable acceptance criteria, and early validation prevent misunderstandings and reduce rework. Therefore, requirement clarity is essential for accurate calculations, reliable software, and improved software quality.

8. A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Introduction:

Boundary value defects occur when software fails to handle input values at the limits of an allowed range. These defects commonly arise when testing focuses only on normal input values and ignores minimum, maximum, and adjacent boundary values. Such defects can cause crashes, incorrect outputs, or unexpected system behaviour.

Classification of the Defect:

1. **Type:** Boundary value defect caused by inadequate boundary testing.
2. **Cause:** Incorrect conditions, improper range validation, or errors in handling minimum and maximum values.
3. **Impact:** The system may crash, reject valid inputs, or accept invalid values.
4. **Testing Gap:** Testers may have tested normal values without checking the limits of the input range.

How Boundary Value Analysis Prevents the Defect:

Boundary Value Analysis (BVA) is a black-box testing technique that focuses on testing values at and around the boundaries of an input range. It helps identify errors that are often missed during normal testing.

1. **Identify Boundaries:** Determine the minimum and maximum permitted input values.
2. **Test Boundary Values:** Check the exact minimum and maximum values.
3. **Test Adjacent Values:** Test values immediately below and above the boundaries.
4. **Test Valid Inputs:** Check values immediately inside the permitted range.
5. **Verify System Behaviour:** Ensure invalid inputs produce appropriate error messages instead of crashes.
6. **Record Results:** Compare actual results with expected results and report any defects found.

Example:

Consider an application that accepts ages from 18 to 60. The tester should check values 17, 18, 19, 59, 60, and 61. The values 18 and 60 must be accepted, while 17 and 61 must be rejected. If entering 60 causes the application to crash, BVA helps identify the boundary-related defect before release.

Benefits of Boundary Value Analysis:

- Detects errors in input validation and range checking.
- Improves test coverage with a small number of test cases.
- Prevents unexpected crashes and incorrect outputs.
- Improves software reliability and user experience.

Conclusion:

Boundary value defects can be prevented through systematic testing of minimum, maximum, and adjacent values. Boundary Value Analysis identifies hidden errors, improves test coverage, reduces failures, and ensures that software handles valid and invalid inputs correctly.

9. A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Introduction:

A defect repository is a centralized database that stores information about software defects identified during testing and after release. It includes defect descriptions, severity, affected modules, root causes, correction details, and testing history. Analysing this information helps teams improve test case design and prevent repeated defects across releases.

1. **Defect Analysis:** Review historical defects to identify frequently affected modules, common errors, and recurring failure patterns.
2. **Test Case Improvement:** Develop new test cases based on previously missed scenarios and update existing test cases to improve coverage.
3. **Regression Testing:** Reuse test cases associated with fixed defects to ensure that the same problems do not reappear in future releases.
4. **Risk-Based Testing:** Prioritize modules that have a history of frequent or critical defects and allocate more testing effort to them.
5. **Root Cause Analysis:** Identify whether defects originated from unclear requirements, design mistakes, coding errors, or inadequate testing.
6. **Knowledge Sharing:** Share defect information among developers and testers to improve understanding and prevent similar mistakes.
7. **Test Suite Maintenance:** Regularly update the test suite using defect trends and lessons learned from previous releases.

Role in Improving Software Quality:

- **Prevents Recurrence:** Ensures previously identified defects are considered during future testing.
- **Improves Coverage:** Adds missing scenarios and strengthens existing test cases.
- **Reduces Cost:** Minimizes repeated debugging, rework, and maintenance expenses.
- **Supports Consistency:** Maintains standardized defect records and corrective actions.
- **Improves Reliability:** Strengthens regression testing and reduces failures across releases.
- **Supports Continuous Improvement:** Helps teams evaluate recurring problems and improve testing strategies.

Example:

An e-commerce application repeatedly experiences duplicate orders when users click the payment button multiple times. By analysing the defect repository, testers identify the recurring problem and create test cases for repeated clicks, slow network responses, payment retries, and interrupted transactions. These tests verify that each order is processed correctly without duplication. The improved test cases can then be reused during regression testing in future releases.

Conclusion:

A defect repository is an important resource for continuous testing improvement. By analysing historical defects, identifying root causes, and updating test cases, teams can prevent repeated failures, improve test coverage, reduce development costs, and deliver reliable, high-quality software.

10. An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

10. Functional Defect in an ATM Withdrawal System

Introduction:

A functional defect occurs when a software feature does not behave according to its specified requirements. In an ATM system, incorrect transaction logic may cause withdrawal failures, incorrect balance updates, or improper cash dispensing. Since banking transactions involve money, functional defects must be detected and corrected before deployment.

Identification of the Functional Defect:

1. **Defect Type:** The issue is a functional defect related to ATM transaction processing.
2. **Cause:** Incorrect implementation of withdrawal logic, balance validation, or transaction confirmation.
3. **Expected Behaviour:** The ATM should verify the account balance, check withdrawal limits, dispense the requested cash, and update the account balance accurately.
4. **Actual Behaviour:** The ATM may deduct money without dispensing cash, dispense cash without updating the balance, or fail during a valid withdrawal.
5. **Impact:** The defect may cause financial losses, incorrect account records, customer complaints, and reduced trust in the banking system.

How the Defect Could Have Been Prevented:

1. **Requirement Analysis:** Clearly define withdrawal rules, balance checks, transaction limits, and failure-handling procedures.
2. **Unit Testing:** Test individual functions responsible for balance validation, withdrawal calculations, and transaction processing.
3. **Integration Testing:** Verify communication between the ATM, banking server, cash dispenser, and account database.
4. **Functional Testing:** Test valid withdrawals, insufficient balances, invalid amounts, and daily withdrawal limits.
5. **Negative and Recovery Testing:** Simulate network failures, power interruptions, and cash dispenser errors to verify safe transaction recovery.
6. **Regression Testing:** Retest withdrawal functions after defect fixes to ensure that existing banking operations remain correct.

Example:

Suppose a customer has ₹5,000 in their account and requests a withdrawal of ₹2,000. The ATM should dispense ₹2,000 and update the balance to ₹3,000. If the balance is deducted but cash is not dispensed due to a transaction logic error, a functional defect has occurred. Proper failure-recovery testing should verify that the transaction is reversed or reconciled correctly without causing financial discrepancies.

Conclusion:

Functional defects in ATM systems can be prevented through clear requirements, systematic functional testing, integration testing, and transaction recovery testing. These practices ensure accurate withdrawals, protect customer funds, maintain data integrity, and improve the reliability and security of banking services.

11. A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Introduction:

Defect clustering is a software testing principle stating that a small number of modules often contain a large proportion of software defects. These modules may have complex logic, frequent modifications, or dependencies on other components. By identifying defect-prone modules, the testing team can prioritize testing activities and use resources more effectively.

Identifying Testing Priorities:

1. **Collect Defect Data:** Review previous test reports and defect repositories to determine the number and severity of defects in each module.
2. **Analyse Defect Distribution:** Identify modules containing a high concentration of defects and examine recurring failure patterns.
3. **Assess Risk and Complexity:** Consider code complexity, frequency of changes, business importance, and the impact of failures.
4. **Prioritize Testing:** Allocate more testing effort to high-risk modules while ensuring that all modules receive adequate testing.
5. **Apply Detailed Testing:** Perform functional, boundary value, integration, and regression testing on defect-prone modules.
6. **Monitor and Update:** Track defect trends after each release and revise testing priorities based on new findings.

How Defect Clustering Improves Testing Efficiency:

- **Better Resource Allocation:** Directs testing effort toward modules where defects are more likely to occur.
- **Higher Defect Detection:** Helps identify critical defects early by focusing on frequently failing components.
- **Reduced Testing Time:** Avoids distributing effort equally across modules with different risk levels.
- **Improved Reliability:** Reduces the likelihood of failures in important and defect-prone components.
- **Reduced Costs:** Detects defects before deployment, minimizing rework and maintenance expenses.
- **Continuous Improvement:** Historical defect analysis helps improve future test plans and test case design.

Example:

In an e-commerce application, defect analysis reveals that most defects occur in the payment and checkout modules. The testing team prioritizes these modules by checking failed payments, duplicate transactions, invalid payment details, interrupted transactions, and incorrect order confirmations. Regression testing is also performed whenever these modules are modified. Other modules, such as product display and search, are tested according to their individual risks and requirements.

Conclusion:

Defect clustering helps identify software modules that require greater testing attention. By analysing historical defect data, assessing risks, and prioritizing defect-prone components, the testing team can improve defect detection, optimize resources, reduce development costs, and deliver more reliable software.

12. A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Introduction:

Defect classification is the process of categorizing software defects based on their nature, severity, impact, and cause. When a web application becomes slow under heavy user load, the issue is classified primarily as a **performance defect** or **non-functional defect** because the system fails to meet expected performance requirements.

Classification of the Defect:

1. **Defect Type:** Performance defect (non-functional defect).
2. **Cause:** Insufficient server resources, inefficient database queries, memory leaks, or poor handling of concurrent requests.
3. **Expected Behaviour:** The application should respond within the specified time while supporting the expected number of concurrent users.
4. **Actual Behaviour:** Response times increase, requests time out, pages load slowly, or the application becomes unavailable.
5. **Severity:** High if the slowdown affects critical operations or prevents users from completing transactions.

Techniques for Identifying the Defect:

1. **Load Testing:** Evaluate application behaviour under the expected number of concurrent users.
2. **Stress Testing:** Increase the load beyond normal limits to identify the breaking point.
3. **Response Time Analysis:** Measure the time taken to complete user requests.
4. **Resource Monitoring:** Examine CPU usage, memory consumption, network traffic, and database performance.
5. **Root Cause Analysis:** Identify whether the problem originates from the application code, server, database, or network.

- **Correct Prioritization:** Helps the team determine the urgency of fixing the problem.
- **Targeted Debugging:** Directs developers toward performance bottlenecks rather than functional logic errors.
- **Effective Resource Allocation:** Helps assign appropriate tools and specialists to resolve the issue.
- **Improved Testing:** Supports the selection of suitable load, stress, and performance tests.
- **Better Reliability:** Helps prevent slowdowns and failures during peak usage.

Example:

An online shopping application works normally with 100 users but becomes very slow when 5,000 users access it simultaneously. Load testing reveals that database queries are taking too long. The defect is classified as a performance defect, and developers optimize the queries and introduce appropriate caching. Further load testing verifies the improvement.

Conclusion:

Defect classification helps identify the nature, cause, severity, and impact of software problems. Correctly classifying a slowdown as a performance defect enables targeted testing and debugging, improves system responsiveness, and ensures reliable operation under heavy user loads.

13. A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Introduction:

Regression testing verifies that modifications, bug fixes, or new features do not negatively affect existing functionality. Historical defect data contains information about previously identified defects, affected modules, root causes, and test cases. Reusing this data helps teams design effective regression tests across multiple releases.

How Historical Defect Data Can Be Reused:

1. **Review Previous Defects:** Analyse defect reports from earlier releases to identify recurring problems and frequently affected modules.
2. **Identify High-Risk Areas:** Prioritize modules that have experienced critical defects or frequent changes.
3. **Reuse Existing Test Cases:** Execute test cases that previously detected defects to verify that those defects have not returned.
4. **Develop Additional Test Cases:** Create new tests for scenarios that were missed during earlier testing.
5. **Automate Regression Tests:** Automate repetitive test cases to improve execution speed and consistency.
6. **Maintain Traceability:** Link historical defects to requirements, affected modules, and corresponding test cases.
7. **Evaluate Results:** Compare current test results with previous results and investigate any newly detected failures.

How It Improves Test Coverage and Efficiency:

- **Better Test Coverage:** Includes previously defective areas and overlooked scenarios.
- **Prevents Defect Recurrence:** Verifies that earlier fixes remain effective in subsequent releases.
- **Reduced Testing Time:** Reuses existing test cases instead of designing every test from scratch.
- **Risk-Based Testing:** Focuses effort on modules with a history of critical defects.
- **Improved Reliability:** Detects unintended effects caused by new changes.
- **Continuous Improvement:** Uses lessons from previous releases to strengthen future test suites.

Example:

An e-commerce application previously experienced duplicate orders when customers retried payments after network interruptions. The team records this defect and its associated test cases in the defect repository. During regression testing of a new release, testers repeat payment retries, interrupted transactions, and repeated button clicks. If duplicate orders occur again, the defect is detected before deployment.

Conclusion:

Historical defect data is a valuable resource for effective regression testing. Reusing previous test cases, analysing recurring defects, prioritizing high-risk modules, and automating repetitive tests improve coverage and efficiency. This approach reduces testing costs, prevents recurring failures, and ensures consistent software quality across releases.

14. During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Introduction:

Integration testing verifies whether different software modules work together correctly. Defects may occur when developers and testers misunderstand requirements, fail to share information, or have different expectations about module interactions. A tester plays an important role in improving communication, identifying defects early, and ensuring that the integrated system meets the specified requirements.

1. **Clear Communication:** Testers should communicate defects clearly by providing steps to reproduce the issue, expected results, actual results, screenshots, and error logs. This helps developers understand and resolve problems quickly.
2. **Requirement Clarification:** Testers should discuss unclear or incomplete requirements with developers, business analysts, and stakeholders. This ensures that all team members understand the expected system behaviour.
3. **Early Defect Reporting:** Testers should report defects immediately after identifying them instead of waiting until the end of the testing phase. Early reporting prevents defects from affecting other modules and reduces the cost of correction.
4. **Continuous Feedback:** Testers should regularly share testing results and defect reports with developers. After developers fix the defects, testers must perform retesting to confirm that the corrections work correctly.
5. **Joint Problem-Solving:** Testers and developers should work together to identify the root causes of defects. A collaborative approach encourages responsibility, reduces misunderstandings, and improves the overall development process.
6. **Test Documentation:** Testers should maintain clear test cases, test data, defect reports, and testing results. Proper documentation ensures that all team members have consistent information and can reproduce problems easily.

Early involvement means including testers during requirement analysis and software design rather than waiting until coding is completed. Testers can identify ambiguous requirements, missing functions, interface mismatches, and possible integration risks at an early stage. They can also prepare test cases and define acceptance criteria before implementation begins.

Finding defects early reduces rework, saves time, and lowers the cost of fixing problems. It also improves test coverage, supports better design decisions, and prevents defects from spreading into multiple modules. As a result, the team can deliver software within the planned schedule with fewer failures.

Example:

In an online shopping application, the payment module may successfully process a payment, but the order module may fail to update the order status. If testers participate in design reviews, they can identify the mismatch between the modules before implementation. Developers can then correct the interface and data exchange requirements, preventing integration failures.

Conclusion:

Testers minimize integration defects through clear communication, early defect reporting, documentation, and collaborative problem-solving. Early involvement helps identify problems before they become expensive to fix, improving software quality, team efficiency, and system reliability.

15. A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Introduction:

Defect prevention is a software quality assurance activity that aims to eliminate the root causes of defects before they occur. When similar defects appear in every release cycle, repeatedly fixing individual problems is not sufficient. The organization must analyse defect patterns, improve development practices, and introduce preventive measures to avoid recurrence.

1. **Root Cause Analysis:** Investigate why defects occur using techniques such as the 5 Whys method and Fishbone Diagram. These techniques help identify the underlying causes rather than simply correcting the visible symptoms. The identified causes should be documented and addressed systematically.
2. **Requirement Reviews:** Requirements should be reviewed carefully to identify ambiguity, missing information, and inconsistencies before development begins. Discussions with stakeholders help ensure that developers and testers share the same understanding of the expected system behaviour.
3. **Coding Standards:** Establish and enforce common coding standards and programming guidelines. Consistent coding practices reduce programming errors, improve readability, and make software easier to maintain. Static code analysis tools can also identify potential defects automatically.
4. **Code Reviews:** Developers should review one another's code before it is merged into the main application. Code reviews help identify logical errors, incorrect conditions, security weaknesses, and violations of coding standards before they affect other modules.
5. **Defect Repository and Documentation:** Maintain a centralized defect repository containing defect descriptions, severity, root causes, corrective actions, and preventive measures. Analysing historical defect information helps the team identify repeated patterns and avoid similar mistakes in future releases.
6. **Training and Knowledge Sharing:** Provide training to developers and testers on common defect types, secure coding, testing techniques, and development tools. Team meetings and lessons-learned sessions help share knowledge and prevent the repetition of known mistakes.
7. **Continuous Monitoring:** Track defect frequency, defect density, recurrence rate, and defect leakage across releases. These measurements help determine whether prevention techniques are effective and identify areas that require further improvement.

How Defect Prevention Improves Overall Software Quality:

Defect prevention reduces recurring errors, debugging time, development costs, and unnecessary rework. It improves software reliability, maintainability, security, and performance. Preventive practices also increase testing efficiency because the team spends less time repeatedly finding and fixing known problems.

Furthermore, fewer defects reach production, reducing system failures and customer complaints. Documentation and continuous improvement help organizations build a consistent quality culture. This leads to more predictable release schedules, improved customer satisfaction, and greater confidence in the software product.

Conclusion:

Defect prevention focuses on eliminating the root causes of software defects rather than repeatedly correcting individual failures. By applying root cause analysis, requirement reviews, coding standards, automated testing, documentation, and continuous process improvement, organizations can reduce recurring defects and deliver reliable, secure, and high-quality software

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)	Marks
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding	
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer	
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application	
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response	
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand	

.